



Introducción a la Programación con R y Python

Antes de comenzar el estudio formal del Cálculo de Probabilidades es necesario familiarizarnos con las herramientas computacionales que utilizaremos a lo largo del diplomado. Nuestro objetivo no es aprender programación de manera exhaustiva, sino adquirir los conocimientos mínimos indispensables para implementar procedimientos matemáticos, realizar cálculos, organizar información y visualizar resultados utilizando R y Python.

A lo largo del curso emplearemos ambos lenguajes dentro del entorno Google Colab, lo que nos permitirá trabajar desde cualquier computadora sin necesidad de realizar instalaciones complejas. Esta modalidad facilitará la experimentación computacional y permitirá concentrarnos en los conceptos probabilísticos que constituyen el eje central del diplomado.

En el Cálculo de Probabilidades necesitamos:

- Trabajar con muchos datos numéricos al mismo tiempo (por ejemplo, miles de simulaciones).
- Realizar cálculos repetitivos de forma rápida y sencilla.
- Visualizar resultados (histogramas, frecuencias, etc.) para entender mejor los fenómenos aleatorios.

¿Qué es programar?

Programar es darle instrucciones claras a la computadora para que realice tareas. En este curso, las instrucciones serán principalmente:

- Guardar números o grupos de números.
- Hacer cálculos con ellos (sumas, promedios, etc.).
- Mostrar los resultados de forma visual.

Piensa en Python o R como calculadoras muy poderosas que puede manejar miles de números a la vez.

Introducción a Google Colab

Antes de escribir nuestro primer código, necesitamos un lugar donde ejecutarlo. Google Colab es como un "cuaderno mágico" que corre en tu navegador. No necesitas instalar nada en tu computadora, solo una cuenta de Google. Esto es ideal porque funciona igual en Windows, Mac, Linux o incluso en una tablet.

Google Colab es una plataforma gratuita que nos permite ejecutar código en la nube sin necesidad de instalar nada en nuestra computadora.

```
In [ ]: # Este es un comentario en Python
print("¡Bienvenidos al Diplomado de Cálculo de Probabilidades!")
```

```
In [ ]: # Este es un comentario en R
print("¡Bienvenidos al Diplomado de Cálculo de Probabilidades!")
```

Variables y tipos de datos

En probabilidad, constantemente trabajamos con números (resultados de experimentos, conteos, proporciones) y también con textos (nombres de eventos, categorías). Las variables son como cajas etiquetadas donde guardamos esta información para usarla después. Por ejemplo, si lanzamos un dado 100 veces, podríamos guardar la cantidad de veces que salió "6" en una variable llamada `seises`.

```
In [ ]: ##### PYTHON

# Variables numéricas
edad = 25
pi = 3.1416

# Cadenas de texto
nombre = "Manuel"
curso = 'Cálculo de Probabilidades'

# Valores lógicos
es_estudiante = True
aprobado = False

print(type(edad))
print(type(nombre))
print(type(es_estudiante))
```

```
In [ ]: ##### R

edad <- 25
pi <- 3.1416
nombre <- "Manuel"
es_estudiante <- TRUE

print(class(edad))
print(class(nombre))
print(class(es_estudiante))
```

Operadores aritméticos y relacionales

Los operadores son las herramientas básicas para hacer cuentas. Los aritméticos (+, -, *, /) los usaremos para sumar probabilidades, promediar resultados, etc. Los relacionales (>, <, ==) nos permiten hacer preguntas como "¿este número es mayor que el umbral?" o "¿estos dos resultados son iguales?".

```
In [ ]: ##### PYTHON

a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a ** 2)      # potencia
print(a % b)       # módulo

print(a > b)
print(a == b)
print(a != b)
```

```
In [ ]: ##### R

a <- 10
b <- 3

print(a + b)
print(a ** 2)      # potencia
print(a %% b)      # módulo
print(a > b)
```

Vectores

En probabilidad rara vez trabajamos con un solo número. Normalmente tenemos muchos números: los resultados de 100 lanzamientos de un dado, las alturas de 50 personas, etc. Un vector es precisamente eso: una colección ordenada de números. Con los vectores podemos calcular el promedio, el máximo, el mínimo o la suma de todos los valores de una sola vez.

```
In [ ]: ##### PYTHON

import numpy as np

edades = np.array([23, 25, 19, 34, 28])
notas = np.array([8.5, 9.0, 7.5, 6.0, 8.0])

print(edades)
print("Promedio:", np.mean(edades))
print("Máximo:", np.max(edades))
```

```
In [ ]: ##### R

edades <- c(23, 25, 19, 34, 28)
notas <- c(8.5, 9.0, 7.5, 6.0, 8.0)

print(edades)
print(mean(edades))
print(max(edades))
```

Indexado de vectores

Cuando trabajamos con vectores que contienen muchos datos (por ejemplo, los resultados de 100 lanzamientos de un dado), frecuentemente necesitamos extraer elementos específicos. Tal vez querremos ver solo el primer resultado, o los últimos 5, o todos aquellos que sean mayores a 4. El indexado es la forma de "apuntar" a posiciones concretas dentro de un vector.

En programación, la mayoría de los lenguajes (incluyendo Python) empiezan a contar desde 0. Esto significa que el primer elemento de un vector está en la posición 0, el segundo en la posición 1, etc. En R, por otro lado, se empieza a contar desde 1. Esta es una de las diferencias más importantes entre ambos lenguajes.

Imagina lo siguiente: Tienes un vector con las calificaciones de 10 estudiantes en un examen. Quieres saber:

- ¿Cuánto sacó el primer estudiante?
- ¿Cuánto sacó el último?
- ¿Cuáles fueron las calificaciones de los estudiantes 3, 5 y 7?
- ¿Qué estudiantes sacaron más de 8?

In []: ##### PYTHON

```
import numpy as np

# Vector con calificaciones de 8 estudiantes
calificaciones = np.array([7.5, 9.0, 6.5, 8.5, 10.0, 5.5, 8.0, 9.5])

print("Vector original:", calificaciones)
print("Longitud del vector:", len(calificaciones))

# Indexado básico (¡recuerda: empieza en 0!)
print("\n--- Indexado básico ---")
print(f"Primer elemento (posición 0): {calificaciones[0]}")
print(f"Segundo elemento (posición 1): {calificaciones[1]}")
print(f"Tercer elemento (posición 2): {calificaciones[2]}")
print(f"Último elemento (posición 7): {calificaciones[7]}")

# Indexado con números negativos (cuentan desde el final)
print("\n--- Indexado con negativos (desde el final) ---")
print(f"Último elemento (posición -1): {calificaciones[-1]}")
print(f"Penúltimo elemento (posición -2): {calificaciones[-2]}")

# Slicing (rebanado) - extraer subconjuntos consecutivos
print("\n--- Slicing (rebanado) ---")
print(f"Elementos del 2 al 4 (posiciones 1 a 3): {calificaciones[1:4]}")
# Nota: [inicio:fin] -> incluye 'inicio', excluye 'fin'

print(f"Elementos desde el inicio hasta la posición 3: {calificaciones[:4]}")
print(f"Elementos desde la posición 4 hasta el final: {calificaciones[4:]}")
print(f"Cada 2 elementos (saltos): {calificaciones[::2]}")

# Indexado con listas de posiciones (elementos no consecutivos)
print("\n--- Indexado con lista de posiciones ---")
posiciones = [0, 2, 4, 6] # posiciones 1ra, 3ra, 5ta, 7ma
print(f"Posiciones {posiciones}: {calificaciones[posiciones]}")

# Indexado booleano (muy útil para filtrar)
print("\n--- Indexado booleano (filtrado por condición) ---")
print(f"Calificaciones mayores a 8: {calificaciones[calificaciones > 8]}")
print(f"Calificaciones menores o iguales a 6: {calificaciones[calificaciones <= 6]}")
print(f"¿Cuántos aprobaron con 7 o más? {sum(calificaciones >= 7)}")

# Ejemplo práctico para probabilidad
print("\n--- Ejemplo: análisis de lanzamientos de dado ---")
lanzamientos = np.array([1, 3, 5, 2, 6, 1, 4, 3, 6, 2, 5, 1])
print(f"Lanzamientos: {lanzamientos}")
print(f"Primer lanzamiento: {lanzamientos[0]}")
print(f"Últimos 3 lanzamientos: {lanzamientos[-3:]}")
print(f"Lanzamientos que fueron 6: {lanzamientos[lanzamientos == 6]}")
print(f"Proporción de unos: {sum(lanzamientos == 1) / len(lanzamientos)}")
```

In []: ##### R

```
# Vector con calificaciones de 8 estudiantes
calificaciones <- c(7.5, 9.0, 6.5, 8.5, 10.0, 5.5, 8.0, 9.5)

print("Vector original:")
print(calificaciones)
print(paste("Longitud del vector:", length(calificaciones)))

# Indexado básico (¡en R se empieza en 1!)
print("\n--- Indexado básico ---")
print(paste("Primer elemento (posición 1):", calificaciones[1]))
print(paste("Segundo elemento (posición 2):", calificaciones[2]))
print(paste("Tercer elemento (posición 3):", calificaciones[3]))
print(paste("Último elemento (posición 8):", calificaciones[8]))

# Indexado con números negativos (excluir posiciones)
print("\n--- Indexado con negativos (excluir) ---")
print(paste("Todos excepto el primero:", calificaciones[-1]))
print(paste("Todos excepto los últimos dos:", calificaciones[-(length(calificaciones)-1)]))

# Slicing (rebanado) - extraer subconjuntos consecutivos
print("\n--- Slicing (rebanado) ---")
print(paste("Elementos del 2 al 4 (posiciones 2 a 4):", calificaciones[2:4]))
# Nota: [inicio:fin] -> incluye 'inicio' y 'fin'

print(paste("Elementos desde el inicio hasta la posición 4:", calificaciones[1:4]))
print(paste("Elementos desde la posición 5 hasta el final:", calificaciones[5:length(calificaciones)]))
print(paste("Elementos en posiciones pares:", calificaciones[c(2,4,6,8)]))

# Indexado con vectores de posiciones (elementos no consecutivos)
print("\n--- Indexado con vector de posiciones ---")
posiciones <- c(1, 3, 5, 7) # posiciones 1ra, 3ra, 5ta, 7ma
print(paste("Posiciones", paste(posiciones, collapse=" "), ":",
  paste(calificaciones[posiciones], collapse=" ")))

# Indexado booleano (muy útil para filtrar)
print("\n--- Indexado booleano (filtrado por condición) ---")
print(paste("Calificaciones mayores a 8:",
  paste(calificaciones[calificaciones > 8], collapse=" ")))
print(paste("Calificaciones menores o iguales a 6:",
  paste(calificaciones[calificaciones <= 6], collapse=" ")))
print(paste("¿Cuántos aprobaron con 7 o más?", sum(calificaciones >= 7)))

# Función which() para encontrar posiciones que cumplen condición
print("\n--- Encontrar posiciones con which() ---")
print(paste("¿En qué posiciones están los 10?",
  paste(which(calificaciones == 10), collapse=" ")))
print(paste("Posiciones de calificaciones > 8:",
  paste(which(calificaciones > 8), collapse=" ")))

# Ejemplo práctico para probabilidad
print("\n--- Ejemplo: análisis de lanzamientos de dado ---")
lanzamientos <- c(1, 3, 5, 2, 6, 1, 4, 3, 6, 2, 5, 1)
print(paste("Lanzamientos:", paste(lanzamientos, collapse=" ")))
print(paste("Primer lanzamiento:", lanzamientos[1]))
print(paste("Últimos 3 lanzamientos:", paste(lanzamientos[(length(lanzamientos)-2):length(lanzamientos)], collapse=" ")))
```

```
print(paste("Lanzamientos que fueron 6:", paste(lanzamientos[lanzamientos == 6], collapse=" "))  
print(paste("Proporción de unos:", sum(lanzamientos == 1) / length(lanzamientos)))
```

Ejercicio 1. Realizaste 20 lanzamientos de un dado y obtuviste los siguientes resultados:

[4,2,6,1,3,5,6,2,4,1,6,3,5,2,4,1,6,3,5,2]

Responde usando indexado:

1. ¿Cuál fue el resultado del primer lanzamiento?
2. ¿Cuál fue el resultado del último lanzamiento?
3. ¿Cuáles fueron los resultados de los lanzamientos 5, 10 y 15?
4. ¿Cuántas veces salió el número 6? (usa indexado booleano)
5. ¿Qué proporción de lanzamientos fueron números pares?

Matrices

A veces nuestros datos tienen dos dimensiones. Por ejemplo, podríamos registrar los resultados de 3 experimentos diferentes, cada uno repetido 4 veces. O podríamos tener las calificaciones de varios estudiantes en diferentes materias. Una matriz es como una tabla de números con filas y columnas. Esto es muy útil para organizar información y para ciertos cálculos de probabilidad que involucran múltiples variables.

```
In [ ]: ##### PYTHON  
  
import numpy as np  
  
M = np.array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])  
  
print(M)  
print("\nDimensión:", M.shape)  
print("Primera fila:", M[0, :])  
print("Segunda columna:", M[:, 1])
```

```
In [ ]: ##### R  
  
M <- matrix(c(1,2,3,4,5,6,7,8,9), nrow=3, byrow=TRUE)  
print(M)
```

Bibliotecas fundamentales

Python y R vienen con funciones básicas, pero para hacer cosas más avanzadas (como operaciones matemáticas complejas, manejo de tablas grandes o gráficos) necesitamos bibliotecas (también llamadas paquetes o librerías). Son como "extensiones" que añaden superpoderes a nuestro

lenguaje. En este curso usaremos principalmente tres en Python: NumPy (para números y vectores), Pandas (para tablas de datos) y Matplotlib (para gráficos). En R usaremos tidyverse, que es una colección de paquetes muy útiles.

```
In [ ]: ##### PYTHON

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

print("Bibliotecas importadas correctamente")
```

```
In [ ]: ##### R

library(tidyverse)

print("Bibliotecas cargadas correctamente")
```

Tablas de datos (Data Frames)

En la vida real, los datos casi nunca vienen como vectores aislados. Normalmente tenemos una tabla donde cada fila es una observación (por ejemplo, un estudiante) y cada columna es una variable (nombre, edad, nota). Esto es exactamente un Data Frame. Aprender a manejar data frames es fundamental porque la mayoría de los problemas de probabilidad aplicada involucran datos organizados de esta forma.

```
In [ ]: ##### PYTHON

import pandas as pd

datos = {
    'Nombre': ['Ana', 'Luis', 'María', 'José'],
    'Edad': [22, 25, 20, 28],
    'Nota': [9.2, 8.5, 7.8, 9.0]
}

df = pd.DataFrame(datos)
df
```

```
In [ ]: ##### R

datos <- data.frame(
  Nombre = c("Ana", "Luis", "María", "José"),
  Edad = c(22, 25, 20, 28),
  Nota = c(9.2, 8.5, 7.8, 9.0)
)

print(datos)
```